Outline

- ✓ Introduction
- ✓ Array
- ✓ Array creation routines
- ✓ Attributes
- ✓ Random Numbers
- ✓ Aggregations, Indexing & Slicing
- ✓ NumPy Arithmetic Operators
- ✓ Sorting array
- ✓ Conditional Selection
- ✓ Beautiful Soup

NumPy

- NumPy (Numeric Python) is a Python library to manipulate arrays.
- Almost all the libraries in python rely on NumPy as one of their main building block.
- NumPy provides functions for domains like Algebra, Fourier transform etc..
- NumPy is incredibly fast as it has bindings to C libraries.
- NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently.
- NumPy is a Python library and is written partially in Python, but most of the parts that require fast computation are written in C or C++.
- Install :
  - ↪ conda install numpy

OR
  - ↪ pip install numpy

Darshan UNIVERSITY

NumPy Array

- The most important object defined in NumPy is an N-dimensional array type called **ndarray**.
- It describes the **collection** of **items** of the **same type**, Items in the collection can be accessed using a **zero-based index**.
- An instance of **ndarray** class can be constructed in many different ways, the basic ndarray can be created as below.

syntax

```
import numpy as np
a= np.array(list | tuple | set | dict)
```

numpyarray.py

```
1 import numpy as np
2 a= np.array(['darshan','Insitute','rajkot'])
3 print(type(a))
4 print(a)
```

Output

```
<class 'numpy.ndarray'>
['darshan' 'Insitute' 'rajkot']
```

Darshan UNIVERSITY

NumPy Array (Cont.)

- **arange**(*start,end,step*) function will create NumPy array starting from *start* till *end* (not included) with specified *steps*.

numpyarange.py

```
1 import numpy as np
2 b = np.arange(0,10,1)
3 print(b)
```

Output

```
[0 1 2 3 4 5 6 7 8 9]
```

- **zeros**(n) function will return NumPy array of given shape, filled with zeros.

numpyzeros.py

```
1 import numpy as np
2 c = np.zeros(3)
3 print(c)
4 c1 = np.zeros((3,3)) #have to give as tuple
5 print(c1)
```

Output

```
[0. 0. 0.]

[[0. 0. 0.] [0. 0. 0.] [0. 0. 0.]]
```

- **ones**(n) function will return NumPy array of given shape, filled with ones.

NumPy Array (Cont.)

- **eye**(*n*) function will create 2-D NumPy array with ones on the diagonal and zeros elsewhere.

numpyeye.py

```
1 import numpy as np
2 b = np.eye(3)
3 print(b)
```

Output

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

- **linspace**(*start,stop,num*) function will return evenly spaced numbers over a specified interval.

numpylinspace.py

```
1 import numpy as np
2 c = np.linspace(0,1,11)
3 print(c)
```

Output

```
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8
0.9 1. ]
```

- **Note:** in **arange** function we have given start, stop & **step,** whereas in **lispace** function we are giving start,stop & **number of elements** we want.

Array Shape in NumPy

- We can grab the shape of ndarray using its **shape property**.

numpyarange.py

```
1 import numpy as np
2 b = np.zeros((3,3))
3 print(b.shape)
```

Output

```
(3,3)
```

- We can also **reshape** the array using reshape method of **ndarray.**

numpyarange.py

```
1 import numpy as np
2 re1 = np.random.randint(1,100,10)
3 re2 = re1.reshape(5,2)
4 print(re2)
```

Output

```
[[29 55]
 [44 50]
 [25 53]
 [59 6]
 [93 7]]
```

- **Note:** the number of elements and multiplication of rows and cols in new array must be equal.
  - **Example :** here we have old one-dimensional array of 10 elements and reshaped shape is (5,2)

    so, 5 * 2 = 10, which means it is a valid reshape

Darshan UNIVERSITY

NumPy Random

- **rand**(p1,p2….,pn) function will create **n-dimensional array** with **random data** using uniform distrubution, if we **do not** specify any parameter it will return **random float** number.

numpyrand.py

```
1 import numpy as np
2 r1 = np.random.rand()
3 print(r1)
4 r2 = np.random.rand(3,2) # no tuple
5 print(r2)
```

Output

```
0.2393725320849050 5

[[0.58924723 0.09677878]
 [0.97945337 0.76537675]
 [0.73097381 0.51277276]]
```

- **randint**(low,high,num) function will create one-dimensional array with ***num*** random integer data between ***low*** and ***high***.

numpyrandint.py

```
1 import numpy as np
2 r3 = np.random.randint(1,100,10)
3 print(r3)
```

Output

```
[78 78 17 98 19 26 81 67 23 24]
```

- We can reshape the array in any shape using **reshape** method, which we learned in previous slide.

NumPy Random (Cont.)

- **randn**(p1,p2….,pn) function will create **n-dimensional array** with **random data** using standard normal distribution, if we **do not** specify any parameter it will return **random float** number.

numpyrandn.py

```
1 import numpy as np
2 r1 = np.random.randn()
3 print(r1)
4 r2 = np.random.randn(3,2) # no tuple
5 print(r2)
```

Output

```
-0.1535961758111037

[[ 0.40967905 -0.21974532]
 [-0.90341482 -0.69779498]
 [ 0.99444948 -1.45308348]]
```

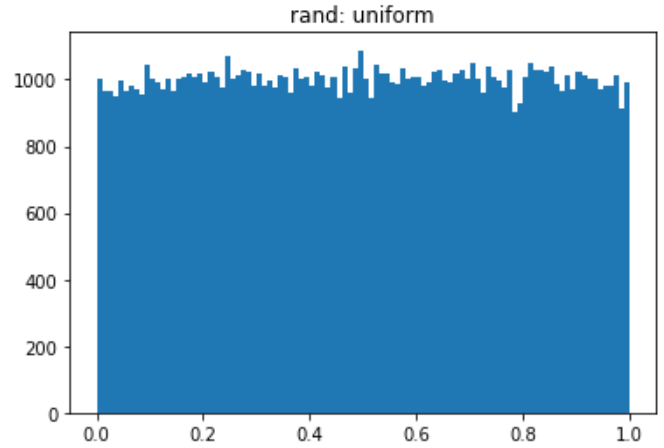
- **Note: rand** function will generate random number using uniform distribution, whereas **randn** function will generate random number using standard normal distribution.
- We are going to learn the difference using visualization technique (as a data scientist, We have to use visualization techniques to convince the audience)

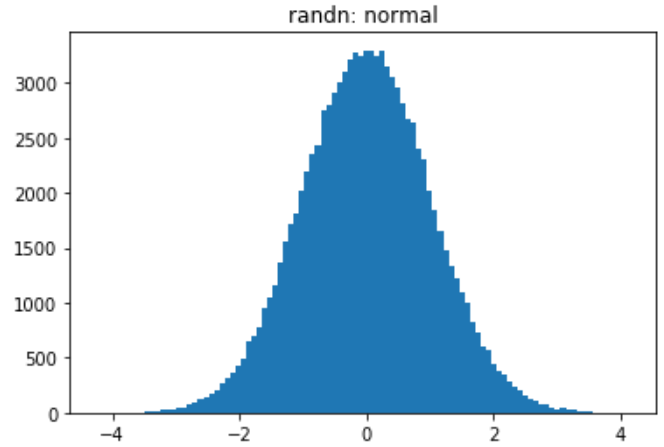
Visualizing the difference between rand & randn

- We are going to use matplotlib library to visualize the difference.
  - You need not to worry if you are not getting the syntax of matplotlib, we are going to learn it in detail in Unit-4

matplotdemo.py

```
1  import numpy as np
2  from matplotlib import pyplot as plt
3  %matplotlib inline
4  samplesize = 100000
5  uniform = np.random.rand(samplesize)
6  normal = np.random.randn(samplesize)
7  plt.hist(uniform,bins=100)
8  plt.title('rand: uniform')
9  plt.show()
10 plt.hist(normal,bins=100)
11 plt.title('randn: normal')
12 plt.show()
```





Darshan UNIVERSITY

Aggregations

- **min**() function will return the minimum value from the ndarray, there are two ways in which we can use min function, example of both ways are given below.

numpymin.py

```
1 import numpy as np
2 l = [1,5,3,8,2,3,6,7,5,2,9,11,2,5,3,4,8,9,3,1,9,3]
3 a = np.array(l)
4 print('Min way1 = ',a.min())
5 print('Min way2 = ',np.min(a))
```

Output

```
Min way1 = 1
Min way2 = 1
```

- **max**() function will return the maximum value from the ndarray, there are two ways in which we can use min function, example of both ways are given below.

numpymax.py

```
1 import numpy as np
2 l = [1,5,3,8,2,3,6,7,5,2,9,11,2,5,3,4,8,9,3,1,9,3]
3 a = np.array(l)
4 print('Max way1 = ',a.max())
5 print('Max way2 = ',np.max(a))
```

Output

```
Max way1 = 11
Max way2 = 11
```

Aggregations (Cont.)

- NumPy support many aggregation functions such as min, max, argmin, argmax, sum, mean, std, etc…

numpymin.py

```
1  l = [7,5,3,1,8,2,3,6,11,5,2,9,10,2,5,3,7,8,9,3,1,9,3]
2  a = np.array(l)
3  print('Min = ',a.min())
4  print('ArgMin = ',a.argmin())
5  print('Max = ',a.max())
6  print('ArgMax = ',a.argmax())
7  print('Sum = ',a.sum())
8  print('Mean = ',a.mean())
9  print('Std = ',a.std())
```

Output

```
Min = 1
ArgMin = 3
Max = 11
ArgMax = 8
Sum = 122
Mean = 5.304347826086956
Std = 3.042235771223635
```

Using axis argument with aggregate functions

- When we apply aggregate functions with multidimensional ndarray, it will apply aggregate function to all its dimensions (axis).

numpyaxis.py

```
1 import numpy as np
2 array2d = np.array([[1,2,3],[4,5,6],[7,8,9]])
3 print('sum = ',array2d.sum())
```

Output

```
sum = 45
```

- If we want to get sum of rows or cols we can use axis argument with the aggregate functions.

numpyaxis.py

```
1 import numpy as np
2 array2d = np.array([[1,2,3],[4,5,6],[7,8,9]])
3 print('sum (cols)= ',array2d.sum(axis=0)) #Vertical
4 print('sum (rows)= ',array2d.sum(axis=1)) #Horizontal
```

Output

```
sum (cols) = [12 15 18]
sum (rows) = [6 15 24]
```

Darshan UNIVERSITY

Trick to understand Axis

Wooden Log

axis= 0
Vertical
Top to Bottom

axis= 1
Horizontal
Across to Log

Single V/S Double bracket notations

- There are two ways in which you can access element of multi-dimensional array, example of both the method is given below

numpybrackets.py

```
1 arr =
2 np.array([['a','b','c'],['d','e','f'],['g','h','i']])
3 print('double = ',arr[2][1]) # double bracket notation
4 print('single = ',arr[2,1]) # single bracket notation
```

Output

```
double = h
single = h
```

- Both method is valid and provides exactly the same answer, but single bracket notation is recommended as in double bracket notation it will create a temporary sub array of third row and then fetch the second column from it.
- Single bracket notation will be easy to read and write while programming.

Slicing ndarray

- Slicing in python means taking elements from one given index to another given index.
- Similar to Python List, we can use same syntax **`array[start:end:step]`** to slice ndarray.
  - Default start is 0
  - Default end is length of the array
  - Default step is 1

numpyslice1d.py

```
1 import numpy as np
2 arr =
  np.array(['a','b','c','d','e','f','g','h'])
3 print(arr[2:5])
4 print(arr[:5])
5 print(arr[5:])
6 print(arr[2:7:2])
7 print(arr[::-1])
```

Output

```
['c' 'd' 'e']
['a' 'b' 'c' 'd' 'e']
['f' 'g' 'h']
['c' 'e' 'g']
['h' 'g' 'f' 'e' 'd' 'c'
'b' 'a']
```

Darshan UNIVERSITY

Array Slicing Example

a =

	C-0	C-1	C-2	C-3	C-4
R-0	1	2	3	4	5
R-1	6	7	8	9	10
R-2	11	12	13	14	15
R-3	16	17	18	19	20
R-4	21	22	23	24	25

- Example :
  - a[2][3] =
  - a[2,3] =
  - a[2] =
  - a[0:2] =
  - a[0:2:2] =
  - a[::-1] =
  - a[1:3,1:3] =
  - a[3:,:3] =
  - a[:,::-1] =

Darshan UNIVERSITY

Slicing multi-dimensional array

- Slicing multi-dimensional array would be same as single dimensional array with the help of single bracket notation we learn earlier, lets see an example.

numpyslice1d.py

```
1 arr =
2 np.array([['a','b','c'],['d','e','f'],['g','h',
  'i']])
3 print(arr[0:2 , 0:2]) #first two rows and cols
4 print(arr[::-1]) #reversed rows
5 print(arr[: , ::-1]) #reversed cols
6 print(arr[::-1,::-1]) #complete reverse
```

Output

```
[['a' 'b']
 ['d' 'e']]
[['g' 'h' 'i']
 ['d' 'e' 'f']
 ['a' 'b' 'c']]
[['c' 'b' 'a']
 ['f' 'e' 'd']
 ['i' 'h' 'g']]
[['i' 'h' 'g']
 ['f' 'e' 'd']
 ['c' 'b' 'a']]
```

Darshan UNIVERSITY

Warning : Array Slicing is mutable !

- When we slice an array and apply some operation on them, it will also make changes in original array, as it will not create a copy of a array while slicing.
- Example,

numpyslice1d.py

```
1 import numpy as np
2 arr = np.array([1,2,3,4,5])
3 arrsliced = arr[0:3]
4
5 arrsliced[:] = 2 # Broadcasting
6
7 print('Original Array = ', arr)
8 print('Sliced Array = ',arrsliced)
```

Output

```
Original Array = [2 2 2 4 5]
Sliced Array = [2 2 2]
```

Darshan UNIVERSITY

NumPy Arithmetic Operations

numpyop.py

```
1  import numpy as np
2  arr1 = np.array([[1,2,3],[1,2,3],[1,2,3]])
3  arr2 = np.array([[4,5,6],[4,5,6],[4,5,6]])
4
5  arradd1 = arr1 + 2 # addition of matrix with scalar
6  arradd2 = arr1 + arr2 # addition of two matrices
7  print('Addition Scalar = ', arradd1)
8  print('Addition Matrix = ', arradd2)
9
10 arrsub1 = arr1 - 2 # substraction of matrix with
   scalar
11 arrsub2 = arr1 - arr2 # substraction of two matrices
12 print('Substraction Scalar = ', arrsub1)
13 print('Substraction Matrix = ', arrsub2)
14 arrdiv1 = arr1 / 2 # substraction of matrix with
   scalar
15 arrdiv2 = arr1 / arr2 # substraction of two matrices
16 print('Division Scalar = ', arrdiv1)
17 print('Division Matrix = ', arrdiv2)
```

Output

```
Addition Scalar =  [[3 4 5]
 [3 4 5]
 [3 4 5]]
Addition Matrix =  [[5 7 9]
 [5 7 9]
 [5 7 9]]
Substraction Scalar =  [[-1  0  1]
 [-1  0  1]
 [-1  0  1]]
Substraction Matrix =  [[-3 -3 -3]
 [-3 -3 -3]
 [-3 -3 -3]]
Division Scalar =  [[0.5 1.  1.5]
 [0.5 1.  1.5]
 [0.5 1.  1.5]]
Division Matrix =  [[0.25 0.4  0.5
]
 [0.25 0.4  0.5 ]
 [0.25 0.4  0.5 ]]
```

NumPy Arithmetic Operations (Cont.)

numpyop.py

```
1  import numpy as np
2  arrmul1 = arr1 * 2 # multiply matrix with scalar
3  arrmul2 = arr1 * arr2 # multiply two matrices
4  print('Multiply Scalar = ', arrmul1)
5  #Note : its not matrix multiplication*
6  print('Multiply Matrix = ', arrmul2)
7  # In order to do matrix multiplication
8  arrmatmul = np.matmul(arr1,arr2)
9  print('Matrix Multiplication = ',arrmatmul)
10 # OR
   arrdot = arr1.dot(arr2)
11 print('Dot = ',arrdot)
12 # OR
13 arrpy3dot5plus = arr1 @ arr2
14 print('Python 3.5+ support = ',arrpy3dot5plus)
```

Output

```
Multiply Scalar =  [[2 4 6]
 [2 4 6]
 [2 4 6]]
Multiply Matrix =  [[ 4 10 18]
 [ 4 10 18]
 [ 4 10 18]]
Matrix Multiplication =  [[24 30
36]
 [24 30 36]
 [24 30 36]]
Dot =  [[24 30 36]
 [24 30 36]
 [24 30 36]]
Python 3.5+ support =  [[24 30 36]
 [24 30 36]
 [24 30 36]]
```

Darshan UNIVERSITY

Sorting Array

- The sort() function returns a sorted copy of the input array.

syntax

```
import numpy as np
# arr = our ndarray
np.sort(arr,axis,kind,order)
# OR arr.sort()
```

Parameters

```
arr     = array to sort (inplace)
axis    = axis to sort (default=0)
kind    = kind of algo to use
('quicksort' <- default,
'mergesort', 'heapsort')
order   = on which field we want
to sort (if multiple fields)
```

- Example :

numpysort.py

```
1 import numpy as np
2 arr =
  np.array(['Darshan','Rajkot','Insitute','of
  ','Engineering'])
3 print("Before Sorting = ", arr)
4 arr.sort() # or np.sort(arr)
5 print("After Sorting = ",arr)
```

Output

```
Before Sorting =  ['Darshan'
'Rajkot' 'Insitute' 'of'
'Engineering']
After Sorting =  ['Darshan'
'Engineering' 'Insitute'
'Rajkot' 'of']
```

Sort Array Example

numpysort2.py

```
1 import numpy as np
2 dt = np.dtype([('name', 'S10'),('age', int)])
3 arr2 =
  np.array([('Darshan',200),('ABC',300),('XYZ',
  100)],dtype=dt)
4 arr2.sort(order='name')
5 print(arr2)
```

Output

```
[(b'ABC', 300) (b'Darshan',
200) (b'XYZ', 100)]
```

Conditional Selection

- Similar to arithmetic operations when we apply any comparison operator to Numpy Array, then it will be applied to each element in the array and a new bool Numpy Array will be created with values True or False.

numpycond1.py

```
1 import numpy as np
2 arr = np.random.randint(1,100,10)
3 print(arr)
4 boolArr = arr > 50
5 print(boolArr)
```

Output

```
[25 17 24 15 17 97 42 10 67
22]
[False False False False
False  True False False  True
False]
```

numpycond2.py

```
1 import numpy as np
2 arr = np.random.randint(1,100,10)
3 print("All = ",arr)
4 boolArr = arr > 50
5 print("Filtered = ", arr[boolArr])
```

Output

```
All =  [31 94 25 70 23  9 11
77 48 11]
Filtered =  [94 70 77]
```

Darshan UNIVERSITY

Web Scrapping using Beautiful Soup

- Beautiful Soup is a library that makes it easy to scrape information from web pages.
- It sits atop an HTML or XML parser, providing Pythonic idioms for iterating, searching, and modifying the parse tree.

webScrap.py

```
1 import requests
2 import bs4
3 req =
  requests.get('https://www.darshan.ac.in/DIET/CE/Faculty')
4 soup = bs4.BeautifulSoup(req.text,'lxml')
5 allFaculty = soup.select('body > main > section:nth-
  child(5) > div > div > div.col-lg-8.col-xl-9 > div >
  div')
6 for fac in allFaculty :
7     allSpans = fac.select('h2>a')
8     print(allSpans[0].text.strip())
```

Output

```
Dr. Gopi Sanghani
Dr. Nilesh Gambhava
Dr. Pradyumansinh
Jadeja
Prof. Hardik Doshi
Prof. Maulik Trivedi
Prof. Dixita Kagathara
Prof. Firoz Sherasiya
Prof. Rupesh Vaishnav
Prof. Swati Sharma
Prof. Arjun Bala
Prof. Mayur Padia
.....
.....
```

Darshan UNIVERSITY